

Multi-tenancy: before and after

What our shared-database multi-tenancy looked like after the first rollout (MT-2 to MT-6), what it looks like now after the hardening work (T1 to T5) and the single-address SaaS work, and why each change was made.

In one paragraph

We still run **one database with shared tables**, every row tagged with its institution. Before, the protection had a weak default: if any code path forgot to say which institution it was acting for, the database showed **every institution's data**. Now that default is reversed (**no institution = no data**), every way into the data has been checked and fixed: background work, partner webhooks, files, caches, unique codes, logs. Reporting tools get their own read-only door, and automated tests fail the build if anything regresses. It all runs on **one shared address** (the SaaS setup) under the restricted database login. The work found and fixed **three real cross-institution leaks** and several features that had quietly stopped working.

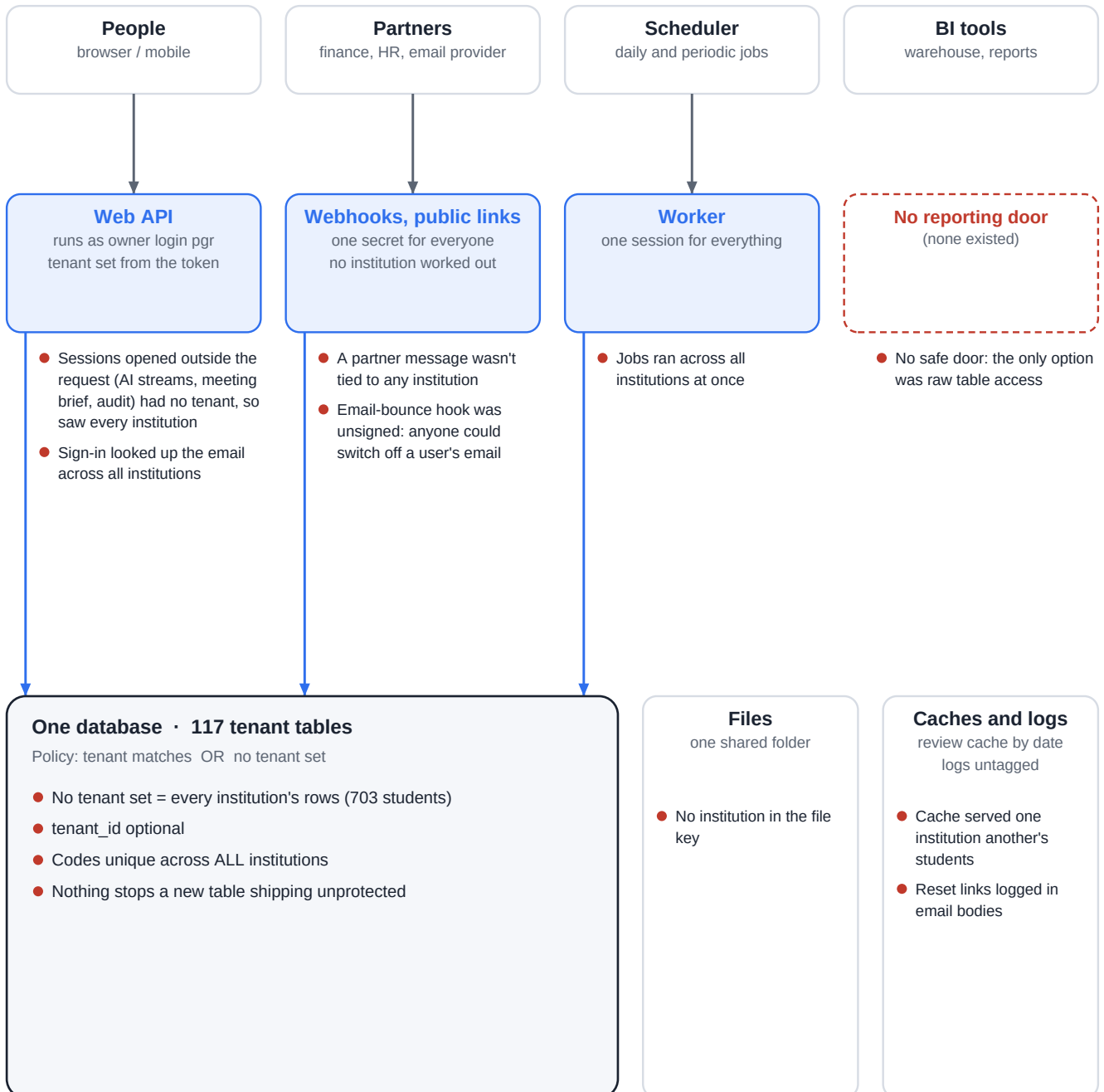
At a glance

	Before	After
If the institution is missing	all 703 students visible	0 rows, writes refused
Tables protected	117 (permissive), 1 without RLS; tenant optional	117 forced, fail-closed; tenant compulsory
Ways into the data	App only; reporting had no safe door	App + a reporting door (240 views, per-institution logins)
Unique codes	Unique across all institutions	Unique within each institution (16 kinds)
Isolation tests	1 test on a scratch table	Guard tests, every GET endpoint swept both ways, reporting-login test
One shared address (SaaS)	Sign-in needed the owner's bypass; restricted login unusable	Works under the restricted login; institution found from the email
Shared HESA specification	Any institution's admin could change it for all	Platform team only
Known leaks	3 live (found by this work)	0 known; fixed and covered by tests

1. Before: the first rollout (MT-2 to MT-6)

One database, row-level security on the tables, and the institution taken from the sign-in token. The red points are where data could leak, or where a feature depended on a weak default.

BEFORE



2. After: hardened, on one shared address

Same shared database and same code, but every path in now establishes its institution before touching data, the database refuses to guess, and reporting has its own door. All of it works on one shared address, with the app on the restricted pgr_app login.

AFTER



3. The single most important change

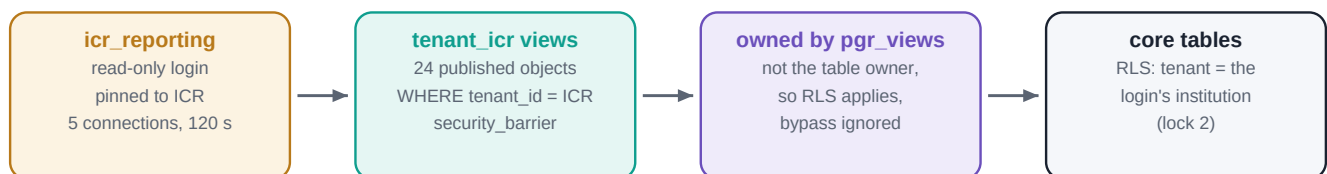
What the database does when code forgets to say which institution it is acting for (measured on the real dev database before and after the upgrade):



Tools that genuinely need every institution (migrations, seeds, backups) now ask for it explicitly with `system_scope()`, and only the table owner may; the app's restricted login `pgr_app` can't, even if code tried.

4. The new reporting door (T3)

The senior developer's view-layer idea, used where it is strongest: each institution gets its own schema of views and its own read-only login. Two locks: the view's fixed filter, and row-level security underneath.



What a reporting login tried, on the real dev database (`test_tenant_views`):

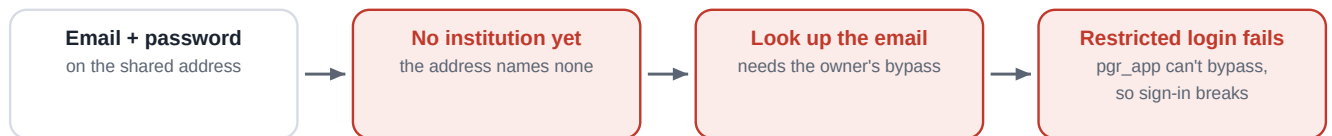
Read its own views	exactly ICR's rows
Read core tables, other schemas, full-detail views	permission denied
SET <code>app.current_tenant = <another institution></code>	0 rows (view filter holds)
SET <code>app.bypass_tenant = 'on'</code>	0 rows (bypass is owner-only)
DELETE / INSERT / UPDATE	refused (read-only)

5. One shared address (the SaaS setup)

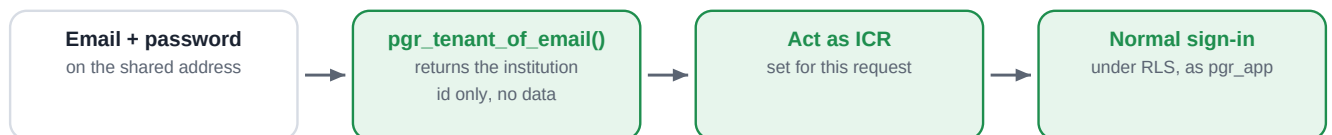
Every institution uses the same address, so the address can't say which institution a request is for. The user does: sign-in emails are unique across institutions, and the sign-in token then carries the institution. Before anyone has signed in, four narrow database functions answer only *which institution is this email, user, referee token or bounced address?* They return ids, never data, and only the app's login can call them.

Signing in

BEFORE

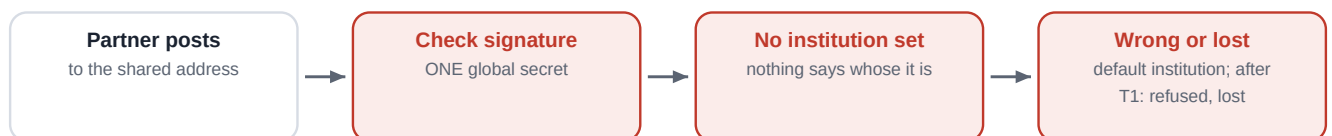


AFTER

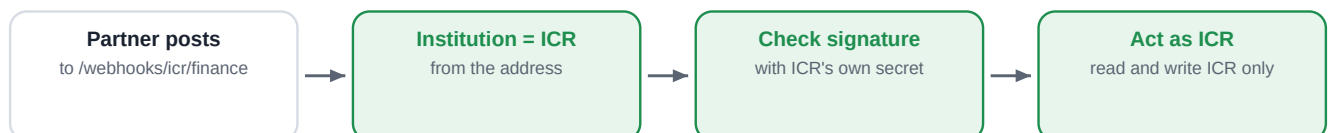


Partner webhooks (and email bounces)

BEFORE



AFTER



The email provider is one account for the whole platform, so a bounce is applied in every institution where that address belongs to a user or person, each in its own scope. The login page no longer lists institutions in production: the email decides.

Proven end to end, with the API on the restricted login (pgr_app) on one address

Check	Result
Sign in, list students, refresh, logout	200 each (25 students returned)
Wrong password, unknown email	401
Password reset request	200
Webhook for Oxbridge with Oxbridge's secret	200, stored under Oxbridge
Webhook for Oxbridge with another institution's secret	401
Webhook for an unknown institution	404
Bounce for an Oxbridge address	Recorded under Oxbridge
Lookup functions	Callable by pgr_app only; leave no bypass behind

6. Real problems found and fixed

Severity	Problem	Fixed in
High	No institution set showed every institution's data	T1
High	AI streams, meeting brief and audit sessions ran with no institution (saw everything)	T1
High	Weekly review cache served one institution another's students	T2
High	Email-bounce hook unsigned: anyone could switch off a user's email	T4
High	Known-password demo accounts created in production on every deploy	T4
High	Password-reset links written to the logs	T4
High	Webhooks and referee submissions silently broken by fail-closed rules	T4
High	On one shared address the restricted login couldn't sign anyone in	SaaS
Medium	Codes unique across institutions; settings savable by only one institution	T4
Medium	Institution admin could grant the vendor-only dev role	T4
Medium	Assistant write intents runnable by other users	T4
Medium	Taught seed export could mix institutions	T4
Medium	Any institution's admin could change the HESA specification for all	SaaS
Medium	Institutions without a partner URL sent to the shared global endpoint	SaaS
Medium	Behind the frontend proxy the backend never saw the user's address	SaaS
Medium	Seed and tool scripts looked records up across every institution	SaaS
Low	Roles page counted users across institutions	T1
Low	Download filename header injection	T4
Low	Public login-page list showed every customer	SaaS
Low	Missing stored file gave a 500	T5
Low	Status-history database tests passed without checking anything	T2

7. What changed, and why

T1 — Close the gaps in the database rules

Change	Before	After	Why
Row-level security policy	One policy: tenant matches, or no tenant set (then every row)	Two policies: <code>tenant_isolation</code> (everyone: tenant matches) and <code>tenant_isolation_system</code> (table owner only: explicit bypass)	A forgotten tenant must mean no data, not all data
Tenant column	Optional on every table	<code>tenant_id</code> NOT NULL on all 117 tables	A row that belongs to nobody can't be protected by a tenant rule
<code>composer_run</code>	No row-level security	Forced RLS like every other table	It was the one unprotected tenant table
Sessions opened outside a request	Tenant set only by the request dependency; AI streams, meeting brief and audit opened their own sessions with none	A global listener publishes the acting tenant on every transaction, whoever opens it	Those sessions saw every institution (a live leak)
Tools that need every institution	Implicit: they just didn't set a tenant	Explicit <code>system_scope()</code> ; migrations opt in for their session	A bypass should be deliberate and visible in code
Background worker	One session across all institutions	Loops over active institutions, one at a time	Under fail-closed rules it would do nothing; and new rows must be stamped with the right institution
Sign-in, refresh, reset	Email looked up across all institutions	Scoped to the subdomain's institution; bare host only uses the bypass for that lookup	An email at another institution should simply be 'not found'
Restricted app login <code>pgr_app</code>	Provisioning rewrote the policies	Grants privileges only; the policies already make it fail-closed	One source of truth for the rules: the migrations
Roles admin page	User counts per role across all institutions	Counts this institution's users only	Small aggregate leak

T2 — Leak tests that guard every future change

Change	Before	After	Why
Definition of 'protected'	Nowhere	app/db/tenant_guard.py: forced RLS, compulsory tenant, exactly the two policies; shared tables listed with a reason	One reviewable rule for every table
New table without a tenant	Nothing would notice	Build fails (model check, no database needed)	A future table can't ship unprotected
Weakened table	Nothing would notice	Guard test fails; proven by deliberately dropping a policy, un-forcing RLS, allowing NULL, adding an open policy	Prove the guard actually catches mistakes
Two-institution sweep	None	Every GET endpoint called as one institution with every permission, both directions; fails on any foreign id or email	Catches leaks anywhere in the API, not just in the database
Fail-closed and pooling tests	None	No tenant = 0 rows in every table; tenant never carries to the next use of a pooled connection, even after an error	The two failure modes that matter most in pooled tenancy
Weekly review cache	Keyed by date only	Keyed by institution and date	Live leak found by the sweep: one institution saw another's students for 30 s
Runner	None	scripts/run_tenant_leak_tests.py runs them on a migrated throwaway copy	Never migrate the working database just to test

T3 — A safe door for reporting and data warehouses

Change	Before	After	Why
Reporting access	None (raw tables were the only option)	Two view schemas per institution: <code>tenant_icr</code> and <code>tenant_icr_full</code>	BI tools need data without touching core tables
What is published	n/a	A catalogue of 24 objects (students, people, lifecycle and HESA leaver data, every dated history, enrolments, funding, supervision, programmes, modules, UOA); views generated from it	One reviewed contract; never hand-written per institution
Personal data	n/a	Standard views: no names, emails, dates of birth (year only) or free text; full detail only on request	Least data by default (GDPR)
Who owns the views	n/a	<code>pgr_views</code> : not the table owner	So row-level security applies and the owner bypass is ignored: a second lock
Reporting logins	n/a	<code><sub>_reporting</code> : read-only, pinned to its institution, 5 connections, 120 s timeout, no core-table rights; created and rotated by a command	Each institution gets only its own door
Keeping in step	n/a	Every migration drops and rebuilds them; new institutions get theirs at creation	Views must never fall behind the schema

T4 — Leak paths outside the database

Change	Before	After	Why
Stored files	One shared folder; key had no institution	Key <institution>/<file>; opening another institution's key reads as not found	A second lock behind the database row
Download filenames	User-supplied name placed raw in the header	Sanitised, with an encoded exact name	Header injection
Institution export / delete	Files missing from exports and left behind on delete	Included and hashed; removed after the rows are committed	A deleted institution should leave nothing
Partner webhooks	One shared secret; no institution	Institution from the subdomain; its own secret	One partner can't post into another institution; and it had silently stopped working after T1
Email-bounce hook	Unsigned and public	Signed and scoped	Anyone could switch off a user's email
Referee submissions	Broken after T1 (no institution)	Institution from the subdomain or the secret token	Feature had quietly stopped working
Unique codes (16 kinds)	Unique across all institutions	Unique within each institution	Two institutions couldn't share a programme code; only one could save a setting; the error revealed others' data
Logs	No institution on any line; email bodies (reset links) logged	Every line tagged; request lines carry the user; no bodies or query strings outside dev	Attribute and separate logs; no secrets or personal data
Demo accounts	Known passwords created on every deploy	Production seed: roles and permissions only	Known credentials in production
Vendor dev role	Any institution admin could grant it	Only a holder can grant it	Privilege escalation
Assistant write intents	Anyone in the institution could run someone's	Only the proposer	Acting for another user

T5 — Ready for an independent security test

Change	Before	After	Why
Test pack	None	docs/security/PEN_TEST_PACK.md: scope, rules, threat model (14 threats with evidence), known issues, before/after checklists	Testers spend time on real risks
Test accounts	None	scripts/pentest_accounts.py: five roles in two institutions, random passwords, refuses production	Cross-institution testing needs two realistic institutions
Missing stored file	Download crashed with a 500	404 'not found'	Found while preparing the test

SaaS — One shared address, and the decisions taken

Change	Before	After	Why
Sign-in before the institution is known	Owner's bypass to find the email; the restricted login couldn't sign anyone in	Four narrow functions (<code>t5_pre_auth_lookups</code>) return only institution ids; callable by <code>pgr_app</code> only	Run the app on the restricted login on one address
Partner webhooks	Institution from the host only	<code>/webhooks/<institution>/<system></code> , signed with that institution's secret	One address serves every institution
Email bounces	Scoped to one institution	Applied in every institution with that address	The provider is one account for the platform
Login-page list	Every customer, publicly	Nothing in production; selector hidden	The customer list isn't public; the email decides
Shared HESA specification	Any institution admin could change it	Platform team only (<code>platform.configure</code>); UI controls hidden for others	One institution can't change the rules for all
Partner URL fallback	Global URL used by any institution without its own	Default deployment only	Messages never go to someone else's partner
Scripts	Seed scripts looked up across all institutions	Act as the default institution	Never pick another institution's record
SLA sweep, alembic current	Sweep unscheduled; read-only commands rebuilt views	Sweep scheduled; only upgrades rebuild	Housekeeping found in the audit
Optional subdomains	Proxy hid the address; links used one URL	<code>TRUST_FORWARDED_HOST</code> ; links to each institution's address	Ready if a client later wants its own address
AI and sign-in emails	—	Left as is, by decision (accepted risk / by design)	Recorded, not changed

8. Evidence

Check	Result
Full backend suite	744 passed; 1 long-standing failure unrelated to tenancy (value-set test)
Tenant tests on a migrated copy	74 passed (guard, sweep both ways, views, catalogue, storage, pre-sign-in lookups)
Restricted login on one address	Sign-in, refresh, logout, reset, webhooks and bounces all work as <code>pgr_app</code> ; no server errors
Dev database after upgrade	At <code>t5_pre_auth_lookups</code> . No tenant: 0 students · default institution: 694 · owner bypass: 703
Real reporting login on dev	Own rows only; core tables, other schemas and writes refused; tenant switch and bypass give 0 rows
Provisioning	<code>pgr_app</code> can't bypass and owns nothing; <code>pgr_views</code> owns 240 views; <code>pgr</code> creates reporting logins
Migrations	Rehearsed on copies (upgrade, downgrade, re-upgrade) before the dev upgrade; backup taken first

9. Still open

No multi-tenancy code is outstanding. What remains needs infrastructure or a client:

- Staging on the restricted login: set APP_DATABASE_URL and APP_ENV=production, then the independent security test using the T5 pack.
- Signed, expiring download links when files move to object storage (S3), and malware scanning of uploads.
- A custom address for a client that wants its own (the subdomain support is already in place).

Accepted and recorded: external AI models receive student data whenever a provider key is set (only the composer has a per-institution switch); sign-in emails stay unique across institutions; reporting logins can see other institutions' schema names, not contents.